

Introduction to Deep Learning with PyTorch

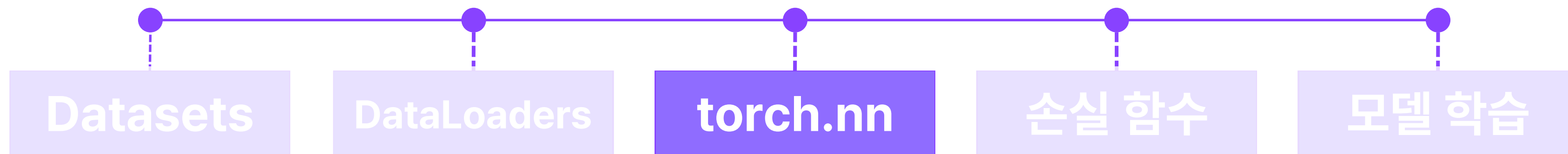
Pytorch로 배우는 딥러닝 기초

PyTorch 모델 구현과 최적화

PyTorch 모델 구현



PyTorch 주요 키워드

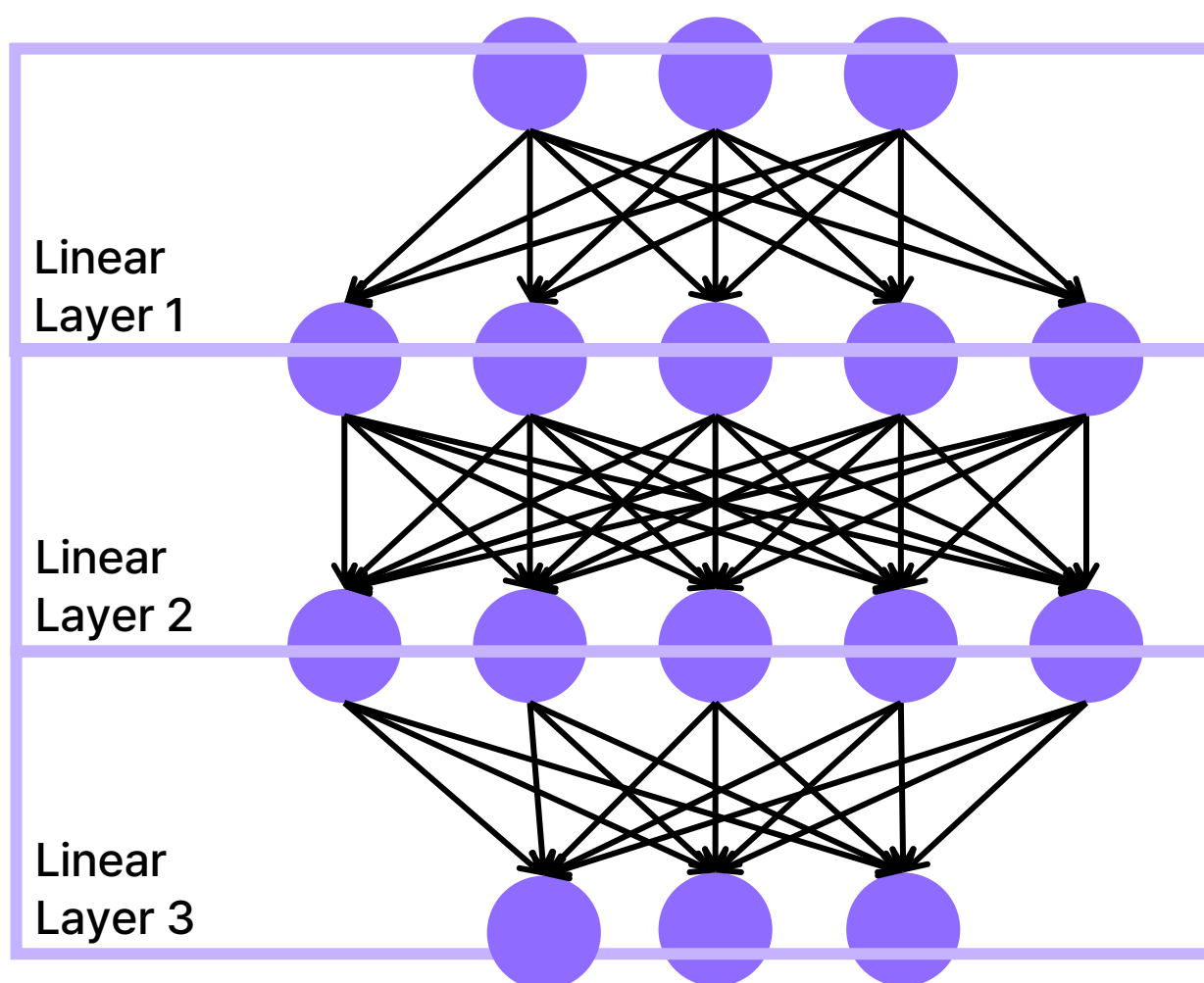
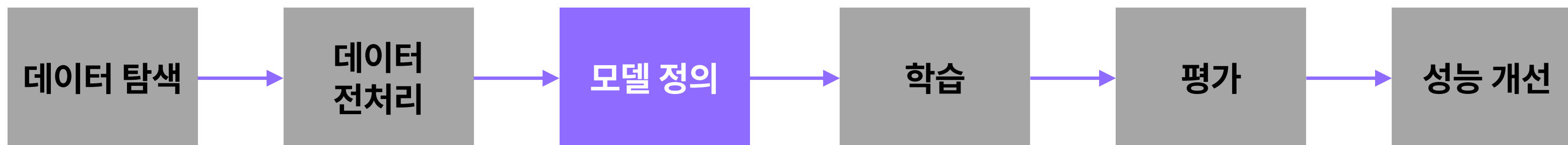


- 딥러닝 모델을 구성하기 위한 여러 레이어가 구현되어 있는 공간
- torch.nn 에 있는 다양한 레이어를 활용해 딥러닝 모델을 구현할 수 있음
- TensorFlow와 비슷하게 Sequential 모델을 활용하여 구현 가능



PyTorch Sequential Model

PyTorch도 TensorFlow와 유사하게 Sequential 모델을 활용하여
PyTorch 레이어를 차례로 나열하는 것 만으로도 모델을 구현할 수 있다.



```
import torch.nn as nn

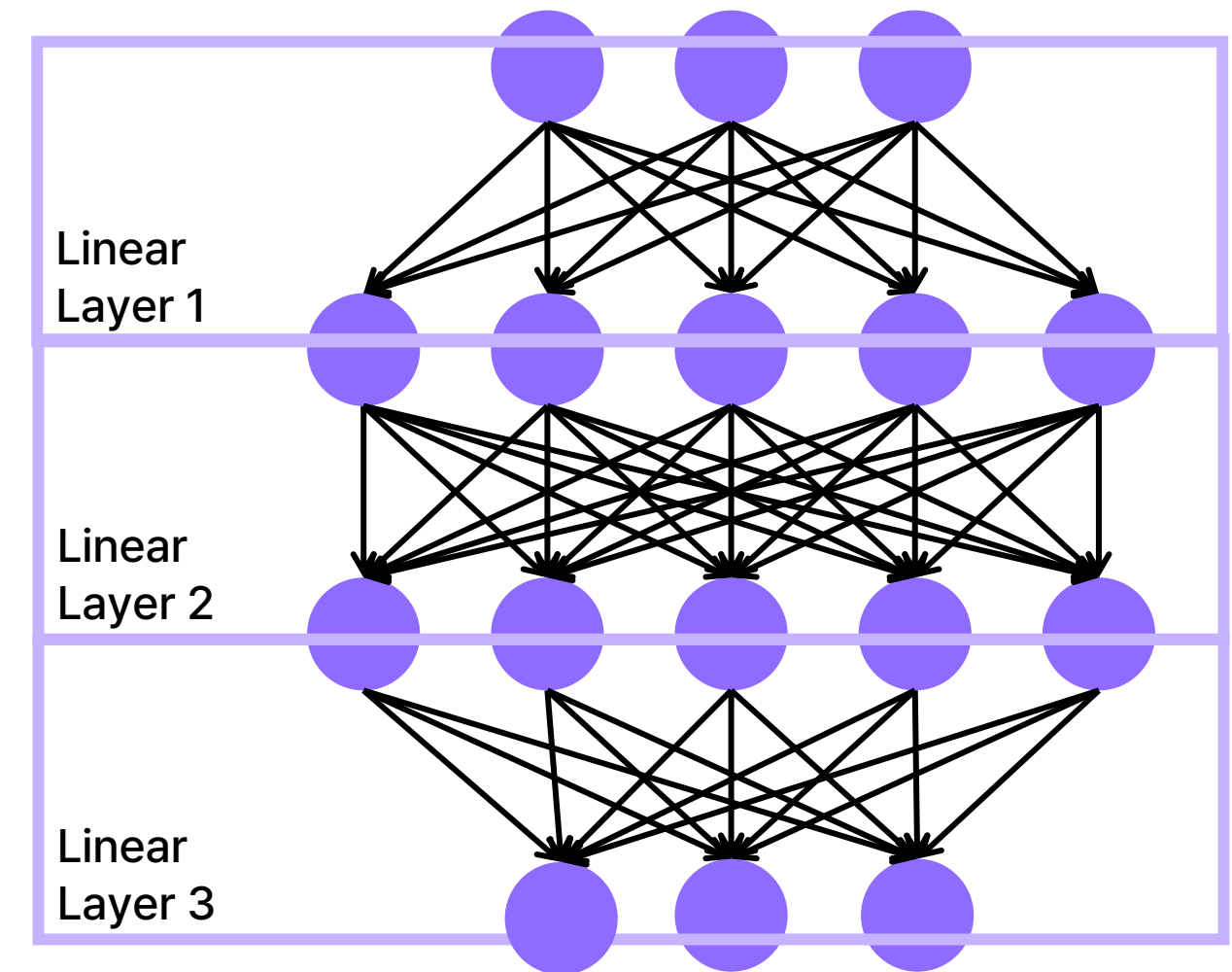
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
    nn.Softmax(dim=1)
)
```

PyTorch Sequential 모델 구현

```
import torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
    nn.Softmax(dim=1)
)
```

1. PyTorch의 Sequential 모델을 생성하고, 원하는 레이어를 일렬로 나열한다.



PyTorch Sequential 모델 구현

```
import torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
    nn.Softmax(dim=1)
)
```

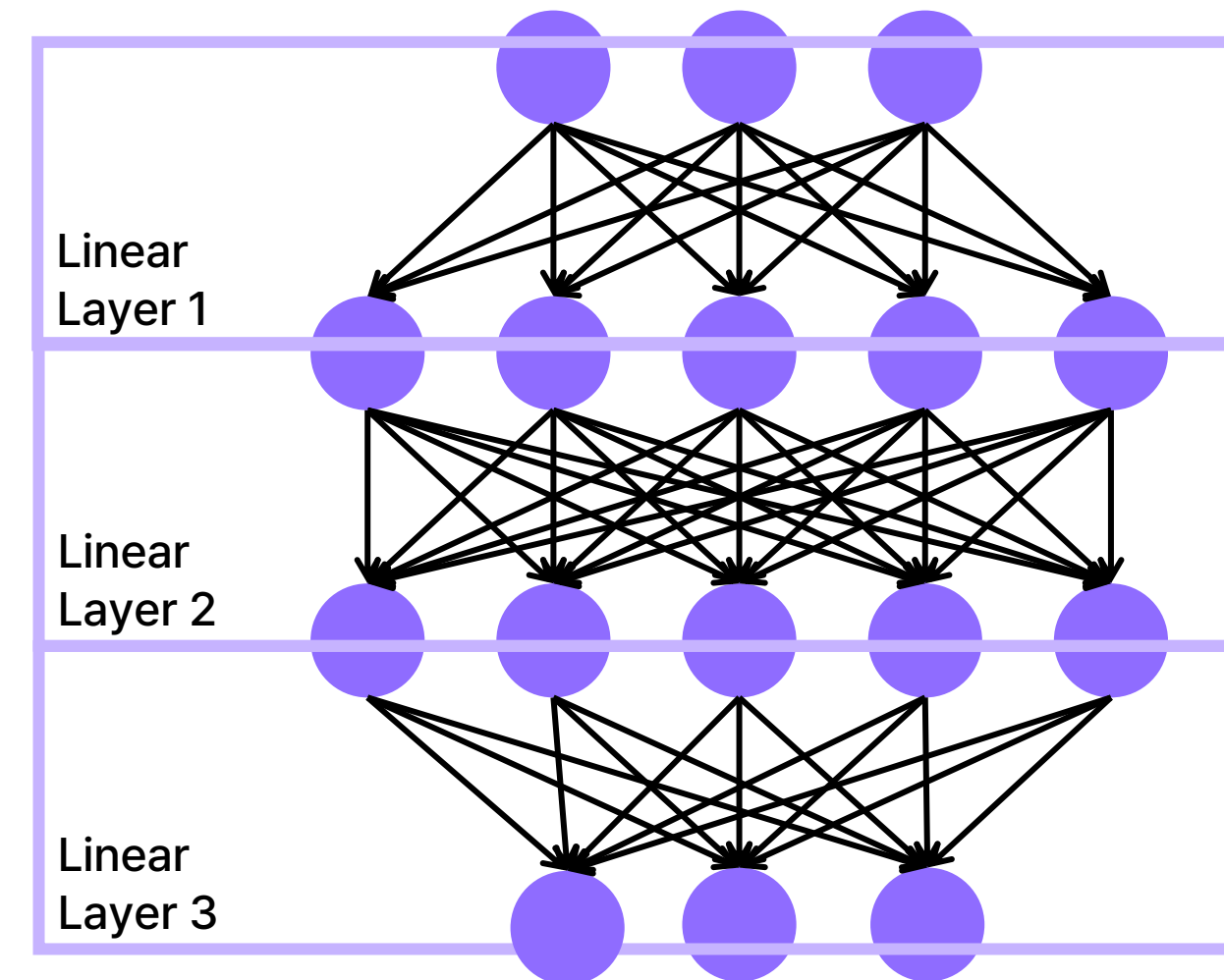
```
model = nn.Sequential()

model.append(nn.Flatten())
model.append(nn.Linear(28 * 28, 128))
model.append(nn.ReLU())
model.append(nn.Linear(128, 64))
model.append(nn.ReLU())
model.append(nn.Linear(64, 10))
model.append(nn.Softmax(dim=1))
```

같은 모델 구조를 생성하는 다른 방법

1. PyTorch의 Sequential 모델을 생성하고, 원하는 레이어를 일렬로 나열한다.

- 이때, 레이어의 **형태**를 지정한다.
- **활성화 함수**는 별개의 레이어로 추가할 수 있다.





PyTorch Linear Layer

```
nn.Linear(28 * 28, 128),
```

- 입력: $28 * 28 = 784$
- 출력: 128
- 가중치 Tensor의 차원: (784, 128)

1. PyTorch의 Sequential 모델을 생성하고, 원하는 레이어를 일렬로 나열한다.
 - 이때, 레이어의 **형태**를 지정한다.
 - **활성화 함수**는 별개의 레이어로 추가할 수 있다.
2. PyTorch Linear Layer는 입력 크기(차원)와 출력 크기(차원)를 모두 설정해야 한다.
 - 가령, 좌측의 예시에서는 입력 데이터의 차원이 784, 출력 데이터의 차원이 128이며, 가중치 Tensor의 차원은 (784, 128)이다.



PyTorch Sequential 모델 구현

```
import torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
    nn.Softmax(dim=1)
)
```

1. PyTorch의 Sequential 모델을 생성하고, 원하는 레이어를 일렬로 나열한다.

- 이때, 레이어의 **형태**를 지정한다.
- **활성화 함수**는 별개의 레이어로 추가할 수 있다.
- TensorFlow와 달리, 별도로 컴파일을 적용하진 않는다.



nn.Module 기반 모델 구현

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.softmax(self.fc3(x))
        return x

model = MLP()
```

1. TensorFlow의 Functional API 처럼, 일렬로 나열하는 것 만으로는 구현하기 어려운 복잡한 모델을 구현할 때 사용할 수 있다.



nn.Module 기반 모델 구현

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.softmax(self.fc3(x))
        return x

model = MLP()
```

1. nn.Module을 상속한 class를 생성하고, __init__ 함수를 생성한다.
 - 필요에 따라 하이퍼파라미터를 인자로 받을 수 있다.



nn.Module 기반 모델 구현

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.softmax(self.fc3(x))
        return x

model = MLP()
```

1. nn.Module을 상속한 class를 생성하고, __init__ 함수를 생성한다.
 - 필요에 따라 하이퍼파라미터를 인자로 받을 수 있다.
2. 모델에 필요한 레이어, 파라미터 등을 설정하는 코드를 구현한다.



nn.Module 기반 모델 구현

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)
```

```
def forward(self, x):
    x = self.flatten(x)
    x = self.relu1(self.fc1(x))
    x = self.relu2(self.fc2(x))
    x = self.softmax(self.fc3(x))
    return x
```

```
model = MLP()
```

1. nn.Module을 상속한 class를 생성하고, __init__ 함수를 생성한다.
 - 필요에 따라 하이퍼파라미터를 인자로 받을 수 있다.
2. 모델에 필요한 레이어, 파라미터 등을 설정하는 코드를 구현한다
3. Forward 함수에서 **입력 데이터에 적용할 연산을 구현한다.**
 - __init__ 함수에서 초기화 한 레이어 등을 활용하여 순전파 과정을 구현한다.



nn.Module 기반 모델 구현

```
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28*28, 128)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.softmax(self.fc3(x))
        return x
```

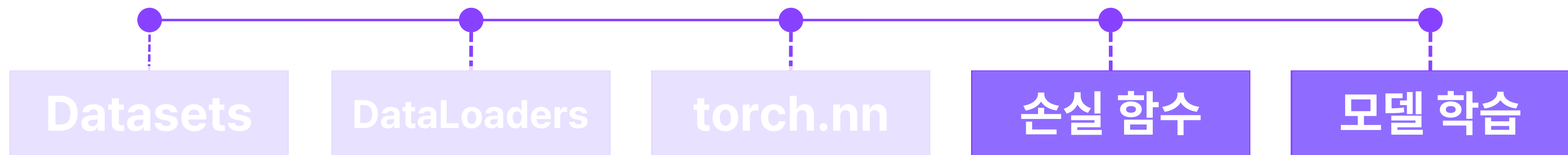
```
model = MLP()
```

1. nn.Module을 상속한 class를 생성하고, __init__ 함수를 생성한다.
 - 필요에 따라 하이퍼파라미터를 인자로 받을 수 있다.
2. 모델에 필요한 레이어, 파라미터 등을 설정하는 코드를 구현한다
3. Forward 함수에서 입력 데이터에 적용할 연산을 구현한다.
 - __init__ 함수에서 초기화 한 레이어 등을 활용하여 순전파 과정을 구현한다.
4. 구현된 class로부터 **모델 객체를 생성한다.**

손실 함수와 모델 최적화



PyTorch 주요 키워드



- TensorFlow 처럼 Adam, SGD를 비롯한 다양한 손실 함수를 모델 학습에 활용 가능
- DataLoader와 손실 함수를 바탕으로 학습



손실 함수와 최적화 함수

torch.nn 에서 MSELoss, CrossEntropyLoss 등
다양한 손실 함수를 필요에 따라 불러와서 사용할 수 있다.

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Loss Functions

nn.L1Loss

nn.MSELoss

nn.CrossEntropyLoss

nn.CTCLoss

nn.NLLLoss



손실 함수와 최적화 함수

torch.optim에서 SGD, Adam 등 다양한 최적화 함수를 불러와서 활용할 수 있다.

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Algorithms

Adadelta

Adafactor

Adagrad

Adam

AdamW



손실 함수와 최적화 함수

이때, PyTorch 모델의 파라미터를 할당하여 해당 파라미터를 최적화 하도록 설정할 수 있다.

Sequential 모델과 nn.Module을 활용한 모델 모두 구분 없이 활용할 수 있다.

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```



최적화 함수 (Optimizer)

손실함수를 활용해 계산한 오차를 최소화하는 방향으로

모델의 가중치를 조정하는 알고리즘

	torch.optim	특징
GD (경사하강법)	(없음)	수학적 기울기(그라디언트)를 활용한 학습
SGD	torch.optim.SGD	Mini-batch를 활용한 효율적인 학습
Momentum	torch.optim.SGD 에서 momentum 인자에 0 초과값을 입력	SGD에 관성을 적용한 빠른 학습
AdaGrad	torch.optim.Adagrad	SGD에 가중치 별 적응형 학습률 적용
RMSprop	torch.optim.RMSprop	AdaGrad에서 현재 정보에 집중
Adam	torch.optim.Adam	Momentum + RMSprop 일반적으로 가장 많이 사용됨

최적화 함수의 학습률은 별도의 인자를 입력하여 설정할 수 있다.

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - 모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - 모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - 모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - **모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.**
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - 모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```

- 모델을 학습 모드로 설정한 후, PyTorch에서 모델을 학습 시키는 과정은 아래와 같다:
 - DataLoader에서 데이터와 라벨을 받아온다.
 - 최적화 함수를 초기화 하고, 모델에 데이터를 입력한다.
 - 모델의 예측값과 라벨값 (참값) 을 바탕으로 손실 (Loss) 을 계산하고, 역전파 연산을 수행한다.
 - 최적화 함수를 통해 모델의 파라미터 (가중치) 를 업데이트 한다.
 - 이 과정을 모든 학습 데이터의 mini-batch에 대해 반복한다

```
epochs = 10
for epoch in range(epochs):
    model.train()
    for images, labels in train_loader: # DataLoader
        optimizer.zero_grad() # 최적화 함수 초기화
        outputs = model(images) # 모델에 데이터를 입력하고 예측값 도출
        loss = criterion(outputs, labels) # 모델의 예측값과 참값을 바탕으로 손실 (Loss) 계산
        loss.backward() # 손실 함수의 역전파 연산
        optimizer.step() # 최적화 함수를 통한 모델 업데이트
```